

04-组件及组件化+组件生命周期

一、组件及组件化

1. 为什么需要组件？

1.1 思考

以可折叠面板为例，现要展示3个，如何操作？

可折叠面板案例的代码：

```
1  <script setup>
2    import { ref } from 'vue'
3    const visible = ref(false)
4
5  </script>
6  <template>
7    <h3>可折叠面板</h3>
8    <div class="panel">
9      <div class="title">
10        <h4>自由与爱情</h4>
11        <span class="btn" @click="visible = !visible"> {{ visible ? '收起' : '展
开' }} </span>
12      </div>
13      <div class="container" v-show="visible">
14        <p>生命诚可贵,</p>
15        <p>爱情价更高.</p>
16        <p>若为自由故,</p>
17        <p>两者皆可抛.</p>
18      </div>
19    </div>
20  </template>
21
22  <style lang="scss">
23    body {
24      background-color: #ccc;
25    }
26
27    #app {
28      width: 400px;
29      margin: 20px auto;
30      background-color: #fff;
```

```

31     border: 4px solid green;
32     border-radius: 1em;
33     box-shadow: 3px 3px 3px rgba(0, 0, 0, 0.5);
34     padding: 1em 2em 2em;
35 }
36 #app h3 {
37     text-align: center;
38 }
39 .panel {
40     .title {
41         display: flex;
42         justify-content: space-between;
43         align-items: center;
44         border: 1px solid #ccc;
45         padding: 0 1em;
46     }
47     .title h4 {
48         line-height: 2;
49         margin: 0;
50     }
51     .container {
52         border: 1px solid #ccc;
53         padding: 0 1em;
54         border-top-color: transparent;
55     }
56     .btn {
57         cursor: pointer;
58     }
59 }
60 </style>

```

1.2 解决方案

有请重量级主角 **组件** 闪亮登场

1. 把需要复用的一段标签, 抽离并封装到一个单独的vue文件里, 连同相关JS和CSS放到一起
2. 哪里要用这个组件, 哪里导入, 当做标签使用即可

2.1、新建文件并填充代码

新建 **src/components/MyPanel.vue** 文件

```

1  <script setup>
2      import { ref } from 'vue'
3      const visible = ref(false)
4  </script>

```

```

5 <template>
6   <div class="panel">
7     <div class="title">
8       <h4>自由与爱情</h4>
9       <span
10         class="btn"
11         @click="visible = !visible">
12         {{ visible ? '收起' : '展开' }}
13       </span>
14     </div>
15     <div
16       class="container"
17       v-show="visible">
18       <p>生命诚可贵,</p>
19       <p>爱情价更高.</p>
20       <p>若为自由故,</p>
21       <p>两者皆可抛.</p>
22     </div>
23   </div>
24 </template>
25
26 <style lang="scss" scoped>
27   .panel {
28     .title {
29       display: flex;
30       justify-content: space-between;
31       align-items: center;
32       border: 1px solid #ccc;
33       padding: 0 1em;
34     }
35     .title h4 {
36       line-height: 2;
37       margin: 0;
38     }
39     .container {
40       border: 1px solid #ccc;
41       padding: 0 1em;
42       border-top-color: transparent;
43     }
44     .btn {
45       cursor: pointer;
46     }
47   }
48 </style>

```

2.2、App.vue导入并使用

```

1  <script setup>
2    // 导入
3    import MyPanel from './components/MyPanel.vue'
4  </script>
5  <template>
6    <h3>可折叠面板</h3>
7    <!-- 使用 -->
8    <MyPanel />
9    <MyPanel />
10   <MyPanel />
11 </template>
12
13 <style>
14   body {
15     background-color: #ccc;
16   }
17
18   #app {
19     width: 400px;
20     margin: 20px auto;
21     background-color: #fff;
22     border: 4px solid green;
23     border-radius: 1em;
24     box-shadow: 3px 3px 3px rgba(0, 0, 0, 0.5);
25     padding: 1em 2em 2em;
26   }
27   #app h3 {
28     text-align: center;
29   }
30 </style>

```

优化完毕了，前后对比一下，小伙伴觉得哪种方式好？

1.3 总结

1. 为什么需要组件？

答：当遇到一段标签(UI)需要复用的时候

2. 如何让这一段标签(UI)复用？

答：抽离 -> 封装(JS+HTML+CSS) -> 导入 -> 使用

2. 组件及组件化

2.1 组件

组件是一个 独立的、可复用 的 Vue 实例，也是一段 独立的 UI 视图 ，代码上体现在是一个独立的 .vue 文件，包含 JS+HTML+CSS 3 部分组成。

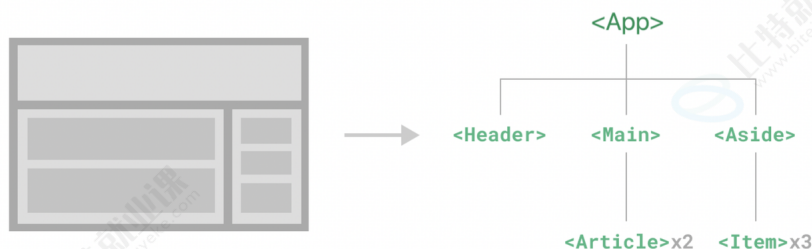
类似乐高和积木一样，我们可以通过任意的乐高或积分进行组合，拼装成我们需要的成品。



2.2 组件化

- 定义：一种代码的开发思想，体现在一个页面可以拆分成一个个组件，每个组件有着自己独立的结构、样式、行为；通过 组件的组合与拼装 形成一个完整的页面，本质是代码的一种拆分思想，化大为小、化繁为简、分而治之
- 好处：各自独立、便于复用

比如：下面这个页面，可以把所有的代码都写在一个页面中，但是这样显得代码比较混乱，难以维护。我们可以按模块进行组件拆分



2.3 总结

1. 什么是组件

答：一个 可复用的、独立的 Vue 实例(UI) ，包含 3 部分代码

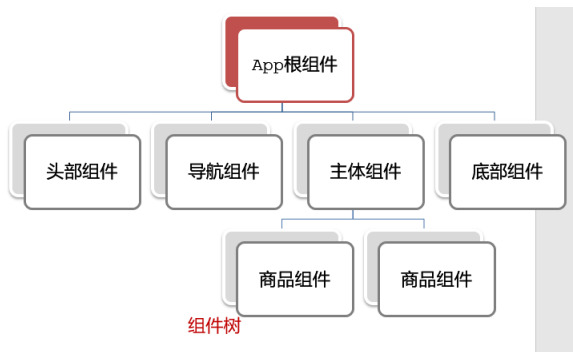
2. 组件化的好处是什么？

答： 化大为小、化繁为简 , 利于代码复用和维护

3. 根组件 App.vue

3.1 根组件

整个应用最上层的组件，包裹所有普通小组件



3.2 组件是由三部分构成

- 三部分构成
 - template: HTML 结构
 - script: JS 逻辑
 - style: CSS 样式 (可支持less/scss, 需要装包)
- 让组件支持less/scss
 - style标签, lang="less/scss" 开启 less/scss 功能
 - 装包: `npm i less less-loader -D` 或者 `npm i sass -D`

3.3 总结

1. App组件我们称之为什么？

答：根组件(顶层组件)

4. 组件的使用

4.1 创建

新建 .vue 文件，编写组件的 3 部分代码

4.2 导入

```
1  import 组件对象 from '相对路径'
2
3  // eg
4  import MyPanel from './components/MyPanel.vue'
```

4.3 注册(仅限于全局组件)

注意: 局部组件无需注册，全局组件要在 `main.js` 中注册

4.4 使用

把组件当做自定义标签使用(单双标签均可)

```

1  <组件名></组件名>
2  <组件名 />
3
4  // eg
5  <!-- 大驼峰标 双标签 -->
6  <MyPanel></MyPanel>
7  <!-- 大驼峰 自闭合的单标签 -->
8  <MyPanel />
9  <!-- 烤串法 双标签 -->
10 <my-panel></my-panel>
11 <!-- 烤串法 自闭合的单标签 -->
12 <my-panel />

```

4.5 案例

在App.vue中使用组件的方式，完成比特就业课官网指定页面布局

<https://bitejiuyeye.com/index>



新建 components/bit-header.vue 组件

```

1  <script setup></script>
2
3  <template>
4    <div class="bit-header">
5      <div class="main">头部内容</div>
6    </div>

```

```

7   </template>
8
9   <style>
10    .bit-header {
11      display: flex;
12      justify-content: center;
13      height: 80px;
14      line-height: 80px;
15      background: #fff;
16      box-shadow: 0 3px 6px 0 rgba(7, 35, 68, 0.15);
17    }
18    .bit-header .main {
19      width: 1200px;
20    }
21  </style>

```

新建 components/bit-body.vue 组件

```

1   <script setup></script>
2
3   <template>
4     <div class="bit-body">
5       <div class="main">
6
7       </div>
8     </div>
9   </template>
10  <style>
11    .bit-body {
12      display: flex;
13      justify-content: center;
14      background: #f6f8fa;
15    }
16    .bit-body h3 {
17      margin-bottom: 15px;
18    }
19    .bit-body .main {
20      width: 1200px;
21    }
22  </style>

```

新建 components/bit-c.vue 组件

代码块


```
1 <template>
2   <div class="bit-c">
3     <h3>C语言</h3>
4   </div>
5 </template>
6
7 <script setup></script>
8
9 <style>
10  .bit-c {
11    padding: 20px 0;
12  }
13 </style>
```

新建 components/bit-ds.vue 组件

代码块

```
1 <script setup></script>
2
3 <template>
4   <div class="bit-ds">
5     <h3>数据结构</h3>
6   </div>
7 </template>
8
9 <style>
10  .bit-ds {
11    margin: 40px 0;
12    padding: 20px 0;
13  }
14 </style>
```

新建 components/bit-java.vue 组件

代码块

```
1 <script setup></script>
2
3 <template>
4   <div class="bit-java">
5     <h3>Java</h3>
6   </div>
7 </template>
8
9 <style>
```

```

10   .bit-java {
11     padding: 20px 0;
12   }
13 </style>

```

新建 components/bit-body.vue 组件

代码块

```

1  <script setup>
2    import BitC from './bit-c.vue'
3    import BitDs from './bit-ds.vue'
4    import BitJava from './bit-java.vue'
5  </script>
6
7  <template>
8    <div class="bit-body">
9      <div class="main">
10        <!-- 烤串法的标签名（推荐，符合W3C的标准，所有标签一律小写） -->
11        <bit-c />
12        <bit-ds />
13        <bit-java />
14
15        <!-- 大驼峰的标签名 -->
16        <!-- <BitC />
17        <BitDs />
18        <BitJava /> -->
19      </div>
20    </div>
21  </template>

```

新建 components/bit-footer.vue 组件

```

1  <script setup></script>
2
3  <template>
4    <div class="bit-footer">
5      <div class="up"></div>
6      <div class="down">
7        <div class="main">底部内容</div>
8      </div>
9    </div>
10 </template>
11
12 <style>

```

```

13   .bit-footer {
14     box-shadow: 0 -1px 3px 0 rgba(0, 0, 0, 0.1);
15     color: #fff;
16   }
17
18   .bit-footer .up {
19     width: 100%;
20     height: 66px;
21     background: #f5f7fd;
22   }
23   .bit-footer .down {
24     display: flex;
25     justify-content: center;
26     height: 200px;
27     line-height: 200px;
28     background: #2d2d3a;
29   }
30   .bit-footer .down .main {
31     width: 1200px;
32   }
33 </style>

```

在 App.vue 中

```

1  <script setup>
2    import BitHeader from './components/bit-header.vue'
3    import BitBody from './components/bit-body.vue'
4    import BitFooter from './components/bit-footer.vue'
5  </script>
6
7  <template>
8    <bit-header />
9    <bit-body />
10   <bit-footer />
11 </template>

```

4.6 总结

1. A组件内部导入组件能在B组件使用吗？

答: no, 不能

2. 使用组件自定义标签 时应该按照什么命名法？

答: 1、大驼峰法，比如 `<BitHeader/>`

2、烤串法，比如 `<bit-header/>`

5. 组件的全局注册

5.1 特点

全局注册的组件，在项目的**任何组件**中都能使用

5.2 步骤

1. 创建.vue组件（三个组成部分）
2. **main.js** 中进行全局注册

5.3 使用方式

当成HTML标签直接使用：

- 双标签： `<组件名></组件名>`
- 自闭合的单标签： `<组件名 />`

5.4 注意

组件名规范：大驼峰命名法或烤串法，如 **BitHeader** 或 **bit-header**

5.5 语法

在 **main.js** 中

```
1  import MyPanel from './components/MyPanel.vue'
2
3  // 注册全局组件
4  // app.component('组件名', 组件对象)
5
6  // 大驼峰组件名
7  app.component('MyPanel', MyPanel)
8
9  // 烤串法组件名
10 app.component('my-panel', MyPanel)
```

5.6 案例

在以下3个组件中展示一个通用的比特课程卡片组件

<https://bitejiuyeke.com/clanguage>

新建 **components/bit-card.vue** 组件

```

1  <script setup></script>
2
3  <template>
4    <div class="bit-card">
5      <div class="img"></div>
6      <div class="desc">
7        <h3>内容标题</h3>
8        <p>主讲人: xxx</p>
9        <p class="exercise">立即练习 &gt;&gt;</p>
10     </div>
11   </div>
12 </template>
13
14 <style>
15   .bit-card {
16     overflow: hidden;
17     display: flex;
18     flex-direction: column;
19     width: 285px;
20     box-shadow: 1px 3px 14px 0 rgba(70, 82, 88, 0.2);
21     border-radius: 6px;
22     cursor: pointer;
23   }
24   .bit-card .img {
25     height: 204px;
26     background: plum;
27   }
28   .bit-card .desc {
29     display: flex;
30     flex-direction: column;
31     padding: 15px 15px 30px 15px;
32   }
33   .bit-card .desc p.exercise {
34     font-size: 14px;
35     text-align: right;
36     color: #2689ff;
37   }
38 </style>

```

在 **main.js** 中，导入并全局注册

```

1  import BitCard from './components/bit-card.vue'
2
3  app.component('BitCard', BitCard)

```

在 bit-c.vue 中

代码块

```
1 <template>
2   <div class="bit-c">
3     <h3>C语言</h3>
4     <bit-card />
5   </div>
6 </template>
```

在 bit-ds.vue 中

代码块

```
1 <template>
2   <div class="bit-ds">
3     <h3>数据结构</h3>
4     <bit-card />
5   </div>
6 </template>
```

在 bit-java.vue 中

```
1 <template>
2   <div class="bit-java">
3     <h3>Java</h3>
4     <bit-card />
5   </div>
6 </template>
```

5.7 总结

1. 全局注册组件应该在哪个文件中注册以及语法是什么？

```
1 // 1. main.js 中注册
2 // 2. 语法:
3
4 // 大驼峰的组件名(推荐)
5 app.component('ComponentName', ComponentObject)
6
7 // 烤串法的组件名
8 app.component('component-name', ComponentObject)
```

2. 全局注册组件在任何一个组件中可不可以用？

答：Yes，一旦注册，任意 .vue 文件中都可用

二、组件生命周期

1. 生命周期介绍

1.1 思考

- 什么时候可以发送初始化渲染请求？（越早越好）
- 什么时候可以开始操作DOM？（至少DOM得渲染出来）



1.2 概念

就是一个Vue实例(组件)从 **创建** 到 **卸载** 的整个过程



1.3 四个阶段

生命周期四个阶段：① 创建 ② 挂载 ③ 更新 ④ 卸载

1. 创建阶段：创建响应式数据
2. 挂载阶段：渲染模板
3. 更新阶段：修改数据，更新视图
4. 卸载阶段：卸载组件

1.4 总结

1. 什么是vue的生命周期？

答：组件从 **创建到卸载** 的过程

2. 组件的生命周期共经历哪几个阶段？

答：4个，创建、挂载、更新、卸载/销毁

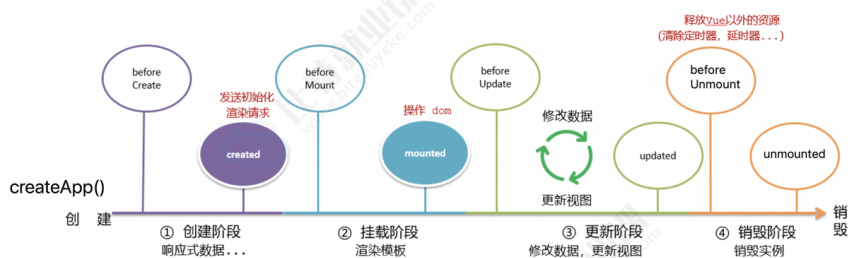
2. 组件生命周期钩子

2.1 介绍

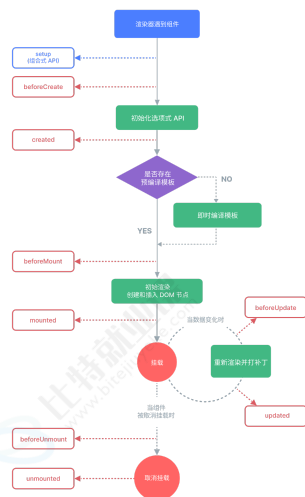
每个 Vue 组件实例在创建时都需要经历一系列的初始化步骤，比如设置好数据监听，编译模板，挂载实例到真实 DOM 树上，以及在数据改变时更新 DOM。在此过程中，会 **自动运行一些函数**，这些函数被称为 **【Vue生命周期钩子】**

2.2 作用

给了开发者在特定阶段添加自己代码的机会



Vue组件整个生命周期官方图示：



2.3 代码示例(选项式API)

新建 `components/LifeCycle.vue`

```
1 <script>
2   export default {
3     // 提供响应式数据
4     data() {
5       return {
6         count: 0
7       }
8     },
```



```

9      // 提供方法/函数
10     methods: {
11         fn() {
12             console.log('fn 函数执行了')
13         }
14     },
15     setup() {
16         console.log('0-setup')
17     },
18     // 创建阶段(第一阶段): Vue组件创建/出生阶段:
19
20     // 创建前: 此时无法访问 data 数据, 也无法调用 methods 方法
21     beforeCreate() {
22         console.log('1-beforeCreate')
23         // console.log(this.count) // undefined
24         // console.log(this.fn) // undefined
25     },
26     // 创建后: 此时可以访问 data 数据, 也可以调用 methods 方法
27     created() {
28         console.log('2-created')
29         // console.log(this.count) // 0
30         // console.log(this.fn) // 访问到函数
31         // this.fn()
32
33         // 开启定时器
34         // 给当前组件实例新增了一个 timerId 属性, 保存了当前定时器的 id 值
35         this.timerId = setInterval(() => {
36             console.log(this.count)
37         }, 1000)
38     },
39
40     // 挂载阶段(第二阶段): 模版渲染阶段
41
42     // 挂载前: 此时写在 template 下的标签还没有变成真实DOM, 故而无法获取DOM
43     beforeMount() {
44         console.log('3-beforeMount')
45         console.log(document.querySelector('p')) // null
46     },
47
48     // 挂载后: 此时写在 template 下的标签已经变成了真实DOM, 故可以获取DOM(是最早可以
    操作DOM的时机)
49     mounted() {
50         console.log('4-mounted')
51         console.log(document.querySelector('p')) // <p>0</p>
52         document.querySelector('p').style.color = 'red'
53     },
54

```

```

55 // 更新阶段(第三阶段): 数据变了, 组件重新渲染的过程
56
57 // 更新前
58 beforeUpdate() {
59   console.log('5-beforeUpdate')
60   // console.log(this.count)
61
62   console.log(document.querySelector('p').innerText) // 旧内容(以前的内容)
63 },
64
65 // 更新后
66 updated() {
67   console.log('6-updated')
68   // console.log(this.count)
69   console.log(document.querySelector('p').innerText) // 新内容
70 },
71
72 // 卸载阶段(第四阶段): 组件移除阶段
73 beforeUnmount() {
74   console.log('7-beforeUnmount')
75 },
76
77 unmounted() {
78   console.log('8-mounted')
79
80   // 关闭定时器
81   clearInterval(this.timerId)
82 }
83 }
84 </script>
85 <template>
86   <div>
87     <p>{{ count }}</p>
88     <button @click="count++">+1</button>
89   </div>
90 </template>
91
92 <style scoped></style>

```

App.vue

```

1 <script setup>
2   import { ref } from 'vue'
3   import Lifecycle from './components/Lifecycle.vue'
4   const isAlive = ref(true)

```

```

5   </script>
6   <template>
7     <life-cycle v-if="isActive"/>
8   </template>

```

2.4 总结

1. 生命周期钩子函数的作用？

答: 钩子函数在特定时机会 **自动执行** , 给了开发者在不同时机有 **添加自己代码** 的机会

2. 选项式API下, 组件首次渲染, 会执行哪几个钩子？

答: 5个, **setup/beforeCreate/created/beforeMount/mounted**

3. 选项式API下, 如果一进入组件就发请求, 在哪个钩子进行？

答: **created**

4. 选项式API下, 最早可以操作原生DOM, 在哪个钩子进行？

答: **mounted**

5. 选项式API下, 组件销毁, 要做优化工作, 在哪进行？

答: **unmounted**

3. 组合式API生命周期钩子

3.1 对比Vue2钩子函数

Vue2钩子函数(选项式) VS Vue3 钩子函数(组合式)

	创建阶段	挂载阶段	更新阶段	销毁阶段
Vue2	beforeCreate created	beforeMount mounted	beforeUpdate updated	beforeUnmount unmounted
Vue3	setup(网络请求)	onBeforeMount onMounted(操作DOM)	onBeforeUpdate onUpdated	onBeforeUnmount onUnmounted(清理工作)

3.2 代码实例

components/LifeCycle.vue

```

1   <script setup>
2     import { onMounted, onUnmounted } from 'vue'
3
4     // 开启定时器

```

```
5     const timer = setInterval(() => {
6         console.log('Hello World')
7     }, 1000)
8
9     // 组件挂载后
10    onMounted(() => {
11        // console.log(document.querySelector('p'))
12
13        // 将 p 标签的字体颜色设置为 green
14        document.querySelector('p').style.color = 'green'
15    })
16
17    // 组件卸载后
18    onUnmounted(() => {
19        // 关闭定时器
20        clearInterval(timer)
21    })
22 </script>
```

3.3 总结

1. 组合式API下, 如果一进入组件就发请求, 在哪个钩子进行?

答: `setup`

2. 组合式API下, 最早可以操作原生DOM, 在哪个钩子进行?

答: `onMounted`

3. 组合式API下, 组件销毁, 要做优化工作, 在哪进行?

答: `onUnmounted`

4. 案例生命周期钩子应用

4.1 在setup中请求数据并渲染



4.1.1 静态代码

```
1 <script setup>
2   import { ref } from 'vue'
3   const i = ref(0)
4   const images = [
5     'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa0f9be4b071da45d9a0c2.jpg',
6     'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa0fa3e4b071da45d9a0c3.jpg',
7     'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa10ace4b071da45d9a0c4.jpg'
8   ]
9
10  const prev = () => {
11    i.value--
12    if (i.value <= -1) {
13      i.value = images.length - 1
14    }
15  }
16
17  const next = () => {
18    i.value++
19    if (i.value >= images.length) {
20      i.value = 0
21    }
22  }
23
24  let timer = null
25
26  const play = () => {
27    timer = setInterval(() => {
28      next()
29    }, 3000)
30  }
31
32  play()
33
34  const stop = () => {
35    clearInterval(timer)
36  }
37 </script>
38
39 <template>
40   <div
41     class="banner"
42     @mouseenter="stop"
43     @mouseleave="play">
44     <ul>
```

```
45     <li
46         v-for="(url, index) in images"
47         :class="{ active: index === i }">
48         
49     </li>
50 </ul>
51 <div class="indicator">
52     <span
53         v-for="(n, index) in images.length"
54         :key="n"
55         :class="{ active: index === i }"
56         @click="i = index">
57     </span>
58 </div>
59 <div class="ctrl">
60     <a
61         href="javascript:;"
62         class="btn prev"
63         @click="prev"
64         >&lt;</a>
65     ><a
66         href="javascript:;"
67         class="btn next"
68         @click="next"
69         >&gt;</a>
70     >
71 </div>
72 </div>
73 </template>
74
75 <style>
76     * {
77         margin: 0;
78         padding: 0;
79     }
80     a {
81         text-decoration: none;
82         color: #000;
83     }
84
85     .banner {
86         position: relative;
87         width: 1200px;
88         height: 337px;
89         margin: 150px auto;
90     }
91     .banner ul {
```

```
92     width: 100%;
93     height: 100%;
94     list-style: none;
95 }
96
97 .banner ul li {
98     position: absolute;
99     top: 0;
100    left: 0;
101    width: 100%;
102    height: 100%;
103    opacity: 0;
104    transition: opacity 1s;
105 }
106 .banner ul li.active {
107     opacity: 1;
108 }
109
110 .banner .indicator {
111     display: flex;
112     justify-content: center;
113     position: absolute;
114     left: 0;
115     bottom: 20px;
116     width: 100%;
117 }
118
119 .banner .indicator span {
120     width: 30px;
121     height: 2px;
122     margin: 0 5px;
123     cursor: pointer;
124     background: rgba(255, 255, 255, 0.6);
125 }
126 .banner .indicator span.active {
127     background: rgba(255, 255, 255, 1);
128 }
129 .btn {
130     position: absolute;
131     top: 50%;
132     width: 30px;
133     height: 30px;
134     line-height: 30px;
135     margin-top: -15px;
136     font-size: 14px;
137     text-align: center;
138     background: rgba(0, 0, 0, 0.3);
```

```
139     color: #fff;
140     border-radius: 50%;
141   }
142   .btn.prev {
143     left: 15px;
144   }
145   .btn.next {
146     right: 15px;
147   }
148 </style>
```

4.1.2 安装 axios 模块

```
1  npm i axios -S
```

4.1.3 请求数据

```
1  <script setup>
2    import axios from 'axios'
3    import { ref } from 'vue'
4    const i = ref(0)
5
6    // 图片列表
7    const images = ref([])
8
9    getBannerData()
10
11    async function getBannerData() {
12      // 发请求, 调接口
13      const resp = await axios({
14        method: 'GET', // 请求方式
15        url: 'http://localhost:4000/api/banner' // 请求路径
16      })
17      // 保存数据
18      images.value = resp.data.data
19    }
20    const prev = () => {
21      i.value--
22      if (i.value <= -1) {
23        i.value = images.value.length - 1
24      }
25    }
26
```



```
27     const next = () => {
28       i.value++
29       if (i.value >= images.value.length) {
30         i.value = 0
31       }
32     }
33   </script>
```

4.2 在onMounted中操作DOM

App组件挂载后，div 盒子上拉收起

4.2.1 静态代码

```
1   <script setup></script>
2
3   <template>
4     <div class="box">我要收起来了</div>
5   </template>
6
7   <style>
8     .box {
9       width: 350px;
10      height: 700px;
11      margin: 50px auto;
12      line-height: 700px;
13      font-size: 27px;
14      color: #fff;
15      text-align: center;
16      background: yellowgreen;
17    }
18    .box.animate {
19      animation: slideUp 4s forwards;
20    }
21
22    @keyframes slideUp {
23      from {
24        height: 700px;
25        opacity: 1;
26      }
27      to {
28        height: 0;
29        opacity: 0;
30      }
31    }
32  </style>
```

4.2.2 完整代码

```
1  <script setup>
2    import { onMounted } from 'vue'
3    onMounted(() => {
4      const div = document.querySelector('.box')
5      div.classList.add('animate')
6    })
7  </script>
```